From Terraform and Terragrunt to Crossplane
A Gradual Migration at Scale



What is Crossplane

Create and manage (cloud) resources with kubectl

MR - Manage Resource is Kubernetes object declares and reconciles one external resource

instance.yaml

```yaml
apiVersion: ec2.aws.upbound.io/v1beta1
kind: Instance
metadata:
  name: payments-worker
spec:
  annotations:
    crossplane.io/external-name: i-0b573*****
  forProvider:
    region: us-west-2
    ami: ami-0123456789abcdef0
    instanceType: t3.small
    tags:
      Name: payments-worker
  deletionPolicy: Delete  # or Orphan
  providerConfigRef:
    name: default
  managementPolicies:
    - Observe
    - Update
    - Delete
    - Update
    - Delete
```

external-name

Create: let the Provider record the EC2 ID.

Import: set the annotation to the exact existing ID, e.g. i-0abc…

deletionPolicy

What happens to the cloud resource when the MR is deleted.

Delete – also destroys the cloud resource.

Orphan – removes the MR, leaves the cloud resource running and intact.

managementPolicies

This explicit list grants Create, Observe, Update and Delete. Remove Delete to keep AWS alive when the MR is removed.

What is a Provider

A Crossplane package that adds new Kubernetes APIs (CRDs) for an external service and runs the controller that reconciles them against that service's API.

provider.yaml

```yaml
apiVersion: pkg.crossplane.io/v1
kind: Provider
metadata:
  name: provider-aws-ec2
spec:
  package: xpkg.upbound.io/upbound/provider-aws-ec2:<pinned-version>
  packagePullPolicy: IfNotPresent
  revisionActivationPolicy: Automatic
  revisionHistoryLimit: 1
  packagePullSecrets:
    - name: registry-creds
  skipDependencyResolution: false
  ignoreCrossplaneConstraints: false
  runtimeConfigRef:
    name: aws-ec2-runtime
```

ProviderRevision

Each package version = one ProviderRevision; it owns the CRDs.

Activation: Automatic (default) or Manual.

RevisionHistoryLimit: Keep N inactive revisions (default 1).

runtimeConfigRef

Points to the DeploymentRuntimeConfig that shapes the provider's pod. Default: **default** — Crossplane always makes sure it exists.

v1

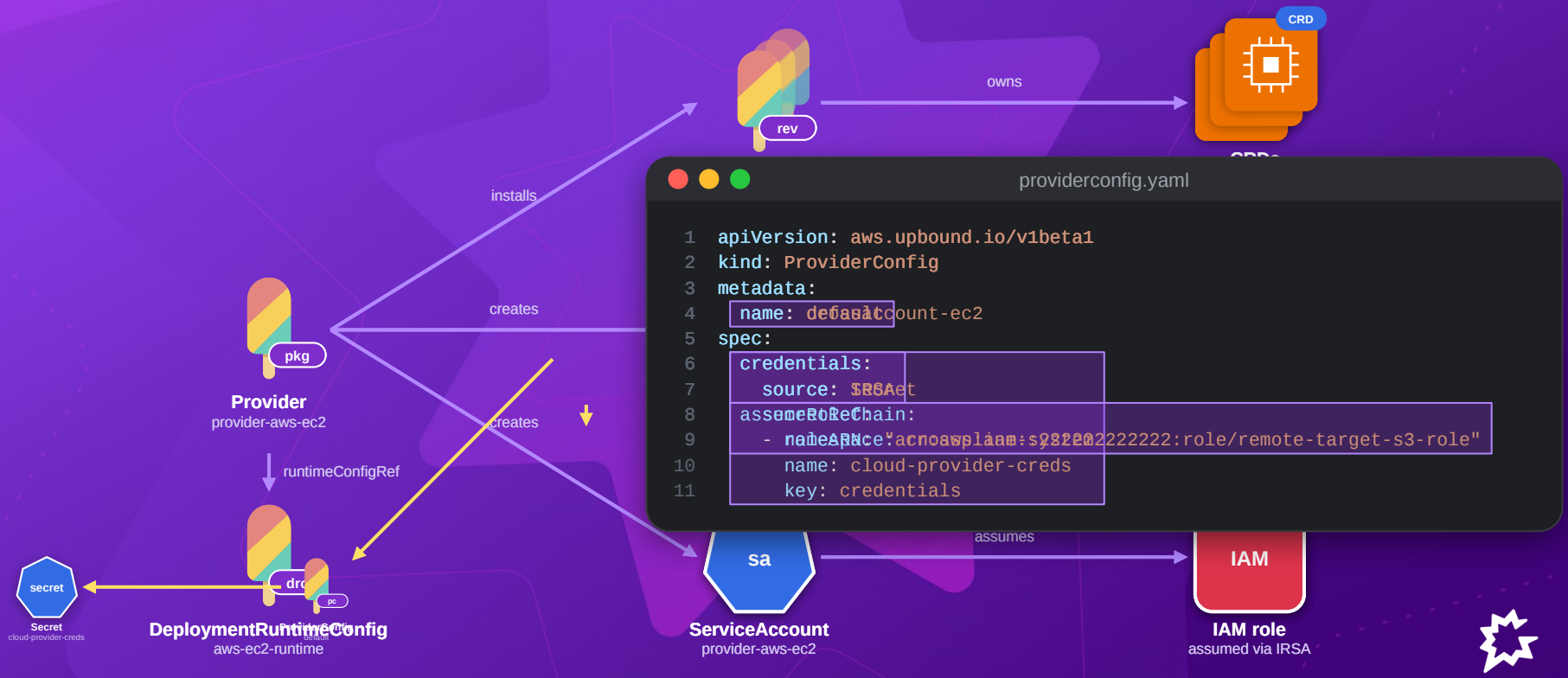
ProviderRevision
(one per version)

CRDs installed by provider-aws-ec2

- Instance
- VPC
- Subnet
- SecurityGroup
- RouteTable
- InternetGateway
- NATGateway
- LaunchTemplate

Provider at runtime

What installing a Provider creates – and how it gets AWS credentials (ProviderConfig)



Provider package

Upbound publishes it · Upjet generates it

provider.yaml

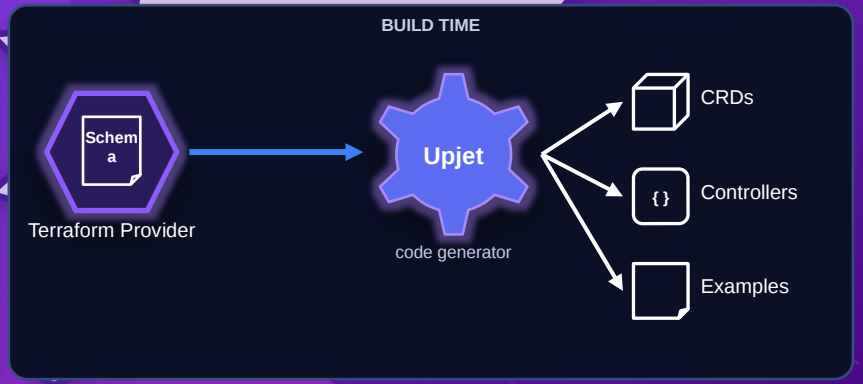
```yaml
apiVersion: pkg.crossplane.io/v1
kind: Provider
metadata:
  name: provider-aws-ec2
spec:
  package: xpkg.upbound.io/upbound/provider-aws-ec2:<pinned-version>
  packagePullSecrets:
    - name: upbound-pull-secret
```

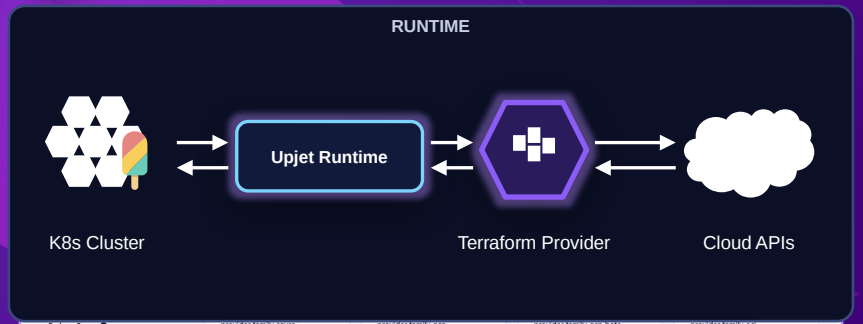
Upbound

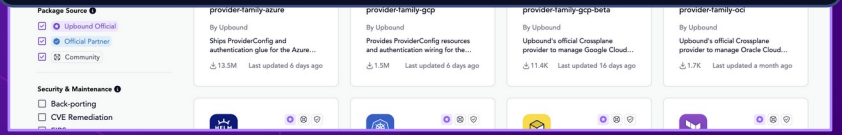
Company behind Crossplane and publisher of official provider packages, tooling and commercial products. Crossplane OSS does not require an Upbound service.

Upjet

Open-source framework that generates Crossplane Provider APIs and controllers from Terraform Provider schemas / logic. Upjet generates the Provider; it is not the Provider installed in the cluster.

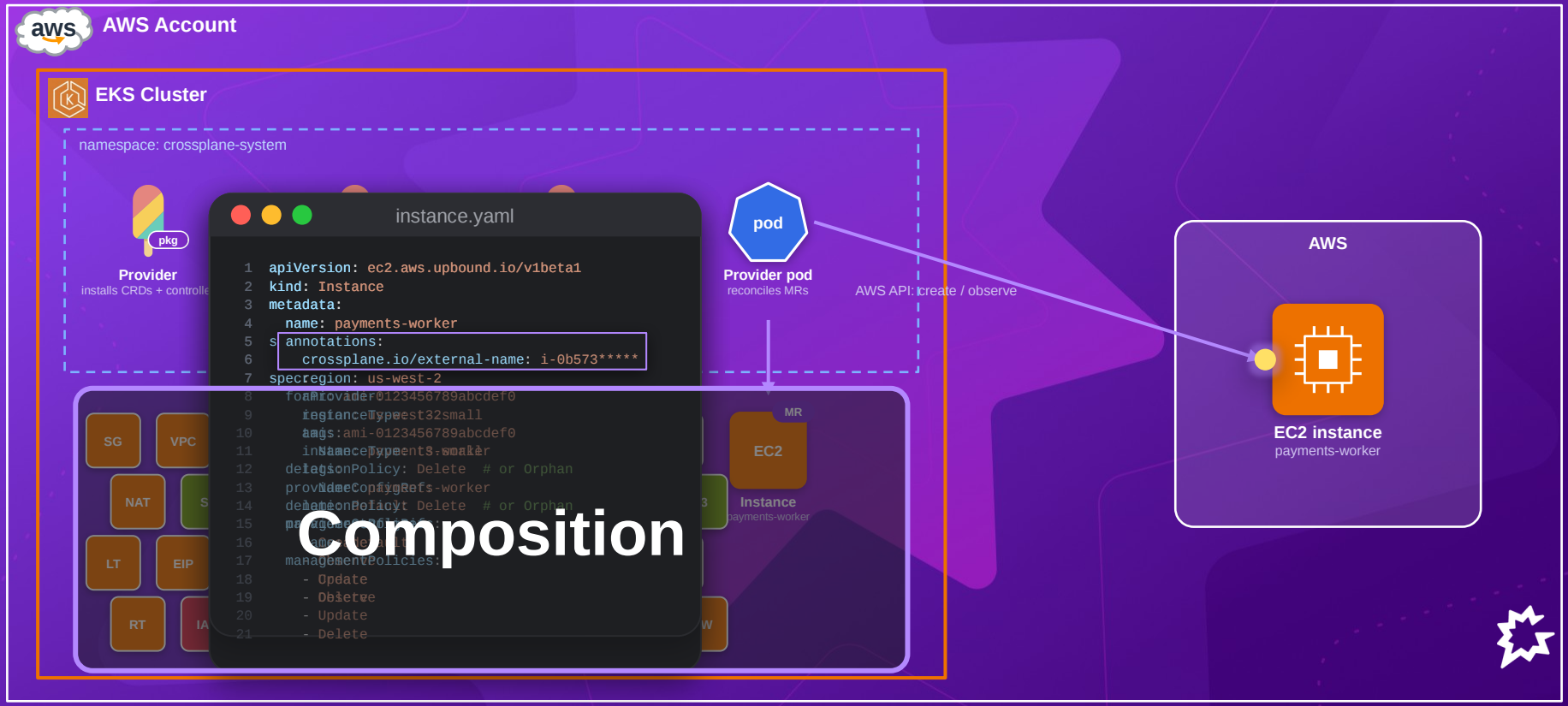






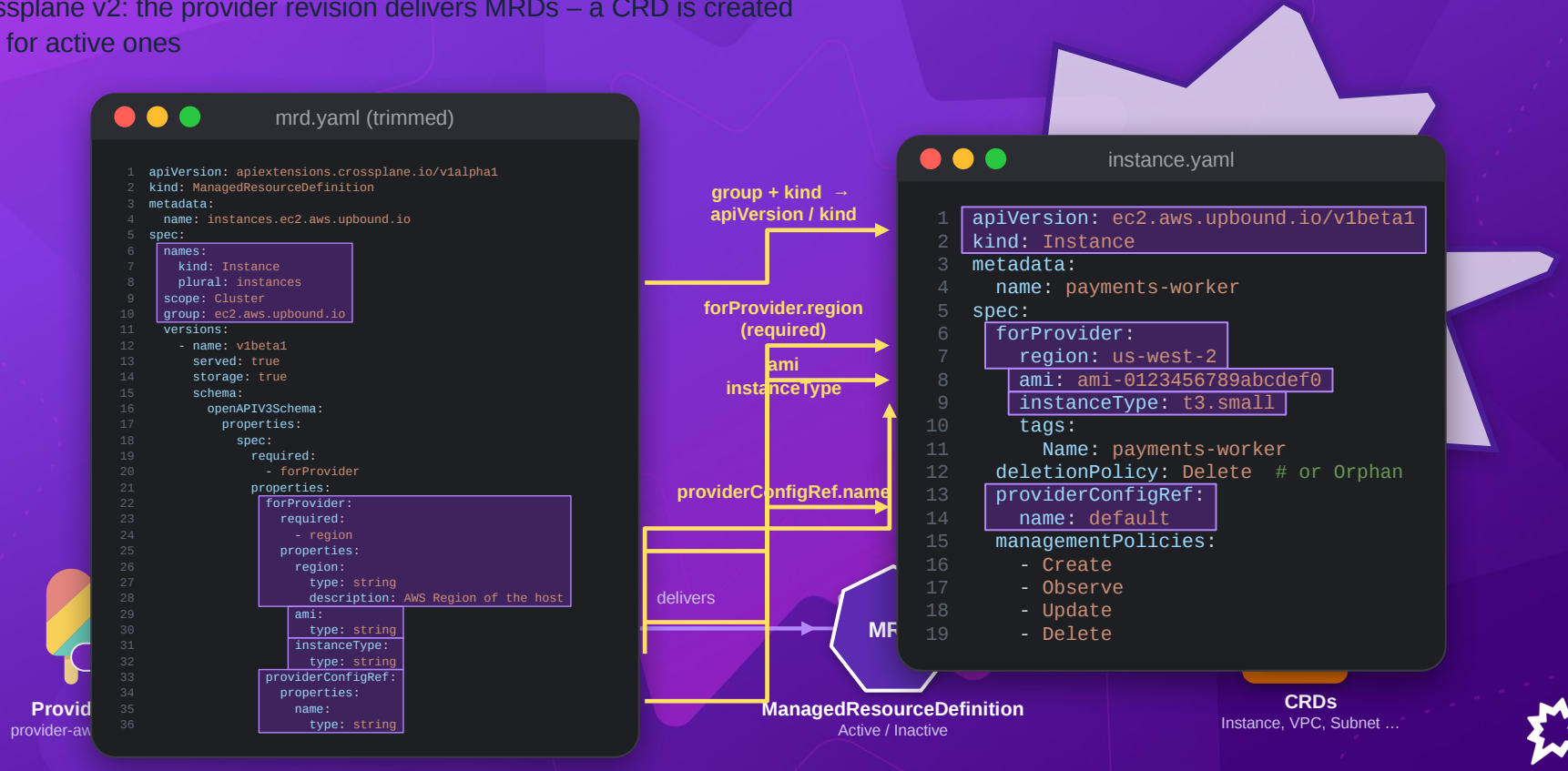
From YAML to one EC2 instance

What runs in the cluster, and how one MR becomes a real instance in the AWS account



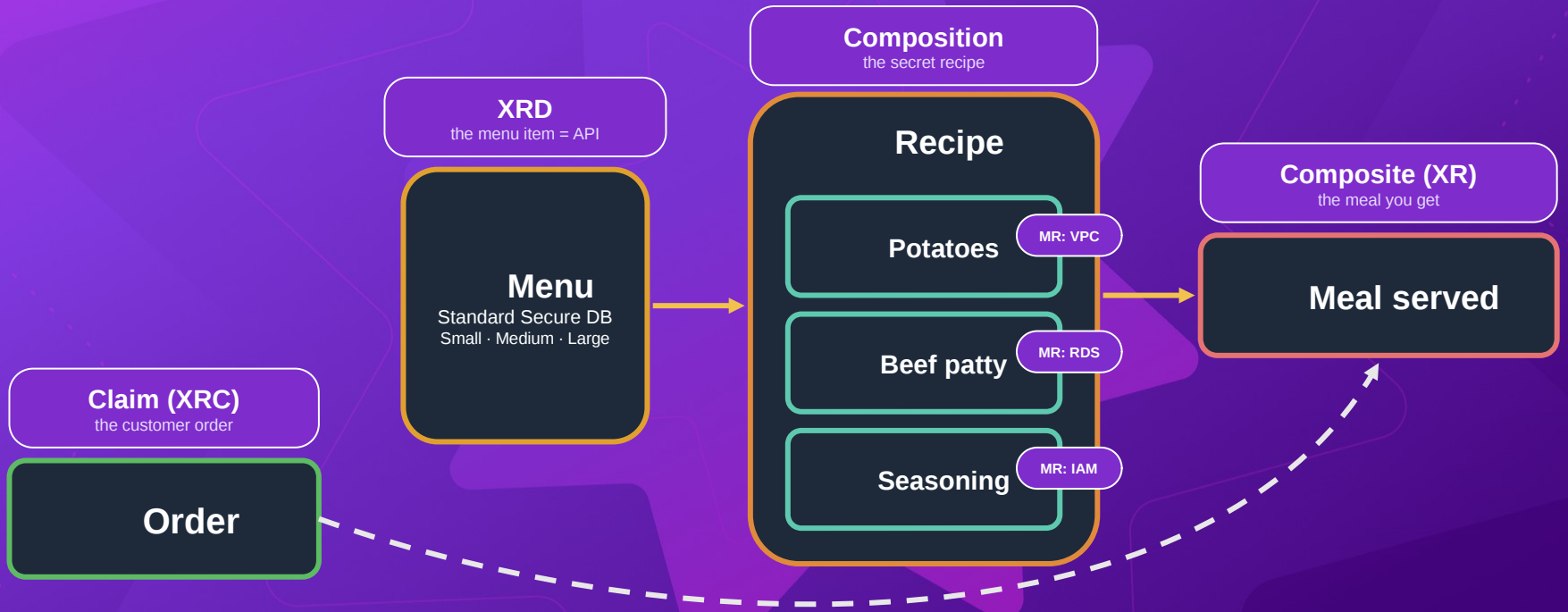
ManagedResourceDefinition (MRD)

Crossplane v2: the provider revision delivers MRDs – a CRD is created only for active ones



What is a Composition?

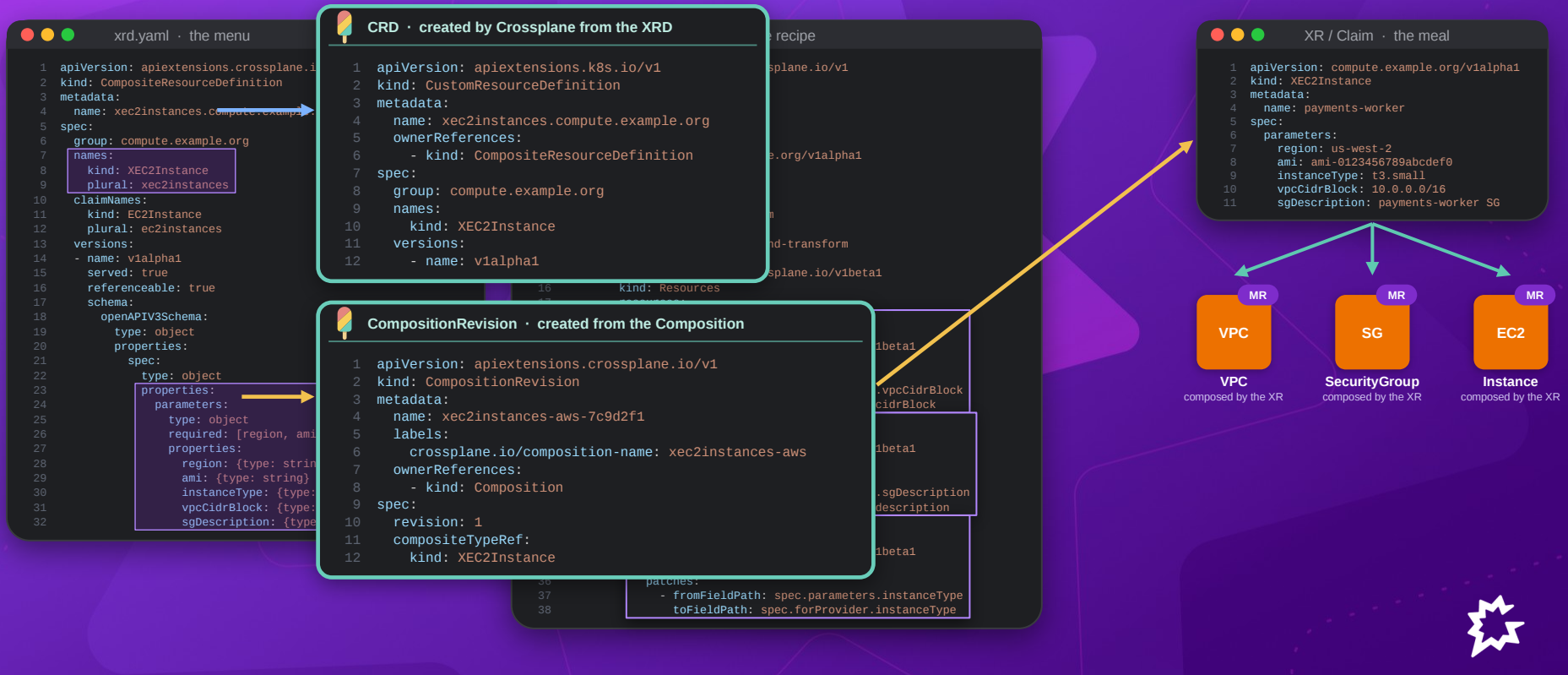
Developers order a meal – they don't buy raw potatoes, frozen patties and cooking oil



Platform engineers write the menu + recipe · developers just order · Crossplane & providers cook

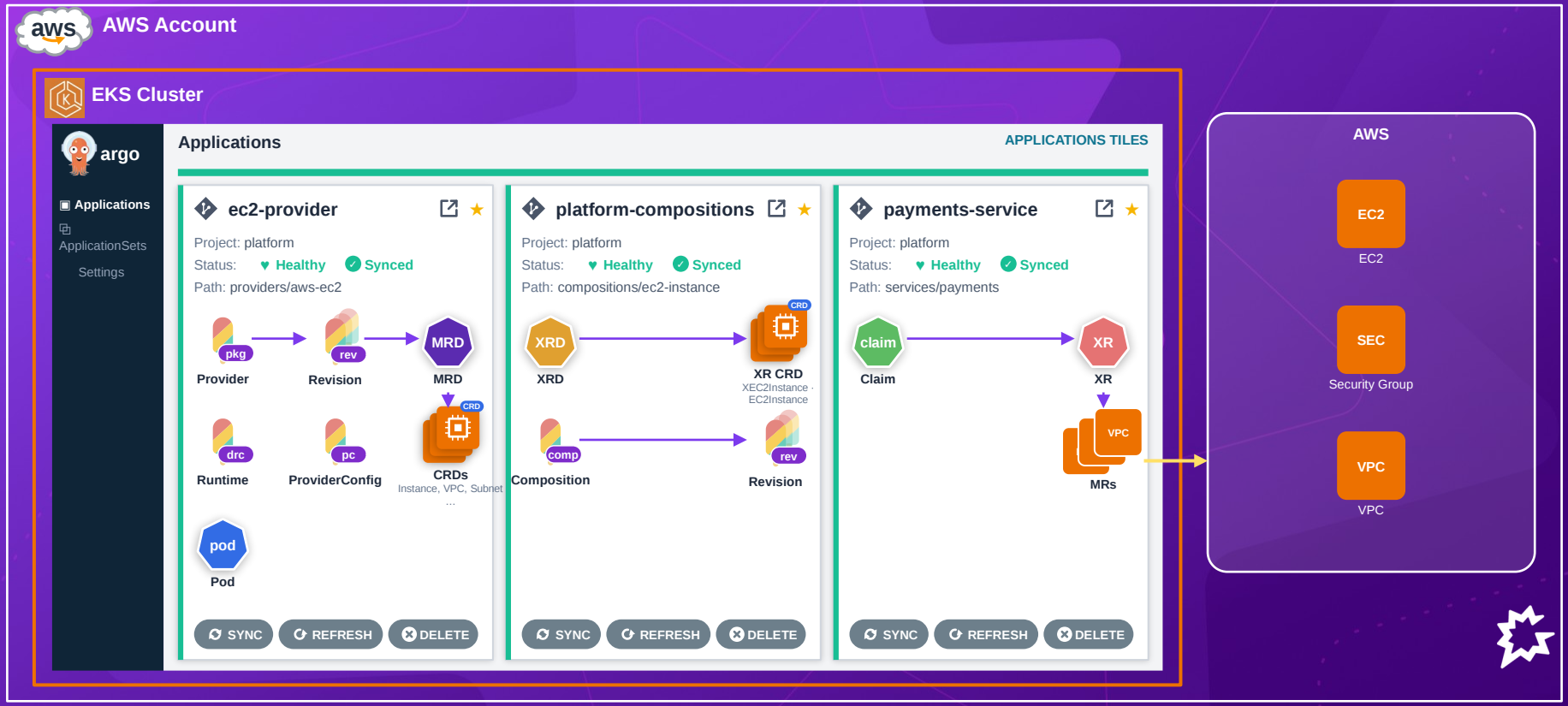
XRD, XR & Claim

The XRD is the API contract; an XR or Claim is a request against that contract



GitOps: Argo CD deploys it all

Three Argo CD apps: the provider, the platform API (XRD + Composition) and a service claim



Why Composition – not a Helm chart of MRs?

Both are GitOps-friendly. Composition adds an API, runtime wiring and one lifecycle.

API contract

The XRD is a typed, validated API (required fields, enums, defaults) – kubectl explain works.

Wiring resources

Values flow between resources at reconcile time (patches / functions read observed state).

One status

One XR: Ready only when every composed resource is Ready.

Lifecycle

Composed resources are owned by the XR – created, updated and deleted as one unit.

Safe rollout

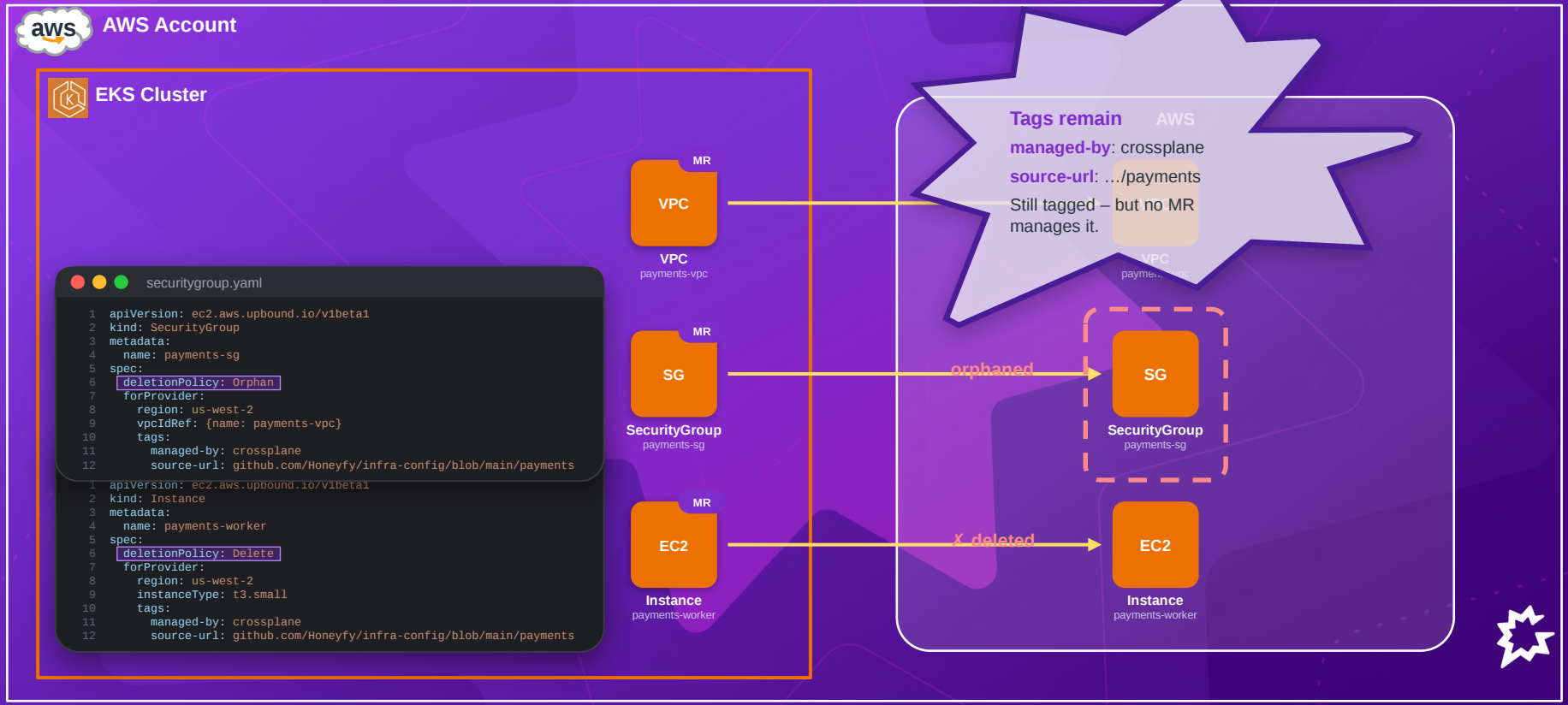
CompositionRevisions – roll forward automatically or pin an XR to a revision (Manual update policy).

Guardrails

The platform hides defaults (tags, IAM, naming) – developers fill a 5-line claim.

deletionPolicy: Delete vs Orphan

What happens in AWS when an MR is deleted?



Finalizers: deletion waits for cleanup

kubectl delete only marks the object – it is removed when every finalizer is gone

securitygroup.yaml

```yaml
apiVersion: ec2.aws.upbound.io/v1beta1
kind: SecurityGroup
metadata:
  name: payments-sg
  deletionTimestamp: "2026-10-15T09:12:00Z"
  finalizers:
    - finalizer.managedresource.crossplane.io
spec:
  deletionPolicy: Orphan
  forProvider:
    region: us-west-2
```

object deleted ✓

1 kubectl delete
The API server only sets deletionTimestamp – the object stays (Terminating).

2 Crossplane provider
deletionPolicy: Orphan → leaves the AWS SG, removes its own finalizer.

3 No finalizers left
Kubernetes removes the object from etcd.



Kyverno: policies as Kubernetes resources

An admission controller – the API server asks Kyverno before it stores anything

add-team-label.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: add-team-label
spec:
  background: false
  rules:
    - name: label-aws-mrs
      match:
        any:
          - resources:
              kinds: ["*"]
              operations: ["CREATE"]
      preconditions:
        all:
          - key: "{{ contains(request.object.apiVersion, 'upbound.io') }}"
            operator: Equals
            value: true
      mutate:
        patchStrategicMerge:
          metadata:
            labels:
              team: platform
```

match

Any kind of object, on **CREATE**.

Only AWS MRs (***.aws.upbound.io**).

mutate

Patch the request: add the label **team: platform**.

Validate	Mutate	Generate	Cleanup
allow / deny a request	change it on the way in	create related objects	delete on a schedule

kubectl / Argo CD → create → **K8s API server** → ask → **Kyverno webhook**

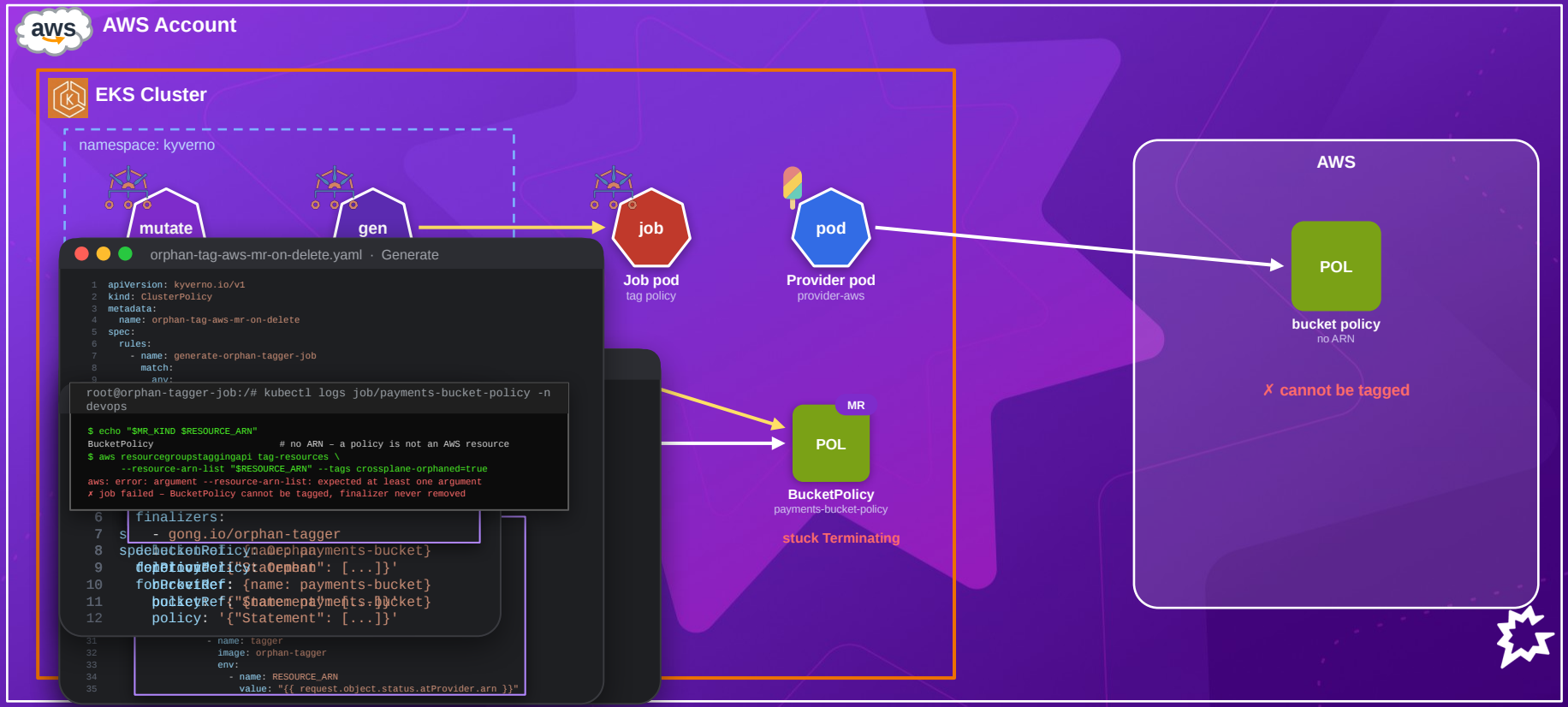
patch

SecurityGroup MR · after Kyverno

```yaml
apiVersion: ec2.aws.upbound.io/v1beta1
kind: SecurityGroup
metadata:
  name: payments-sg
  labels:
    team: platform      # added by Kyverno
```

Kyverno Mutate + Generate: our first orphan tagger

Mutate adds the finalizer on CREATE – Generate starts a tagging Job when the MR is deleted



Taggable vs non-taggable: the CRD decides

The MRD creates the CRD – the orphan-tagger-watcher reads each CRD schema to decide if a kind is taggable



instance-crd.yaml — ✓ taggable = true

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: instances.ec2.aws.upbound.io
  ownerReferences:
    - kind: ManagedResourceDefinition
spec:
  group: ec2.aws.upbound.io
  names: {kind: Instance}
  versions:
    - name: v1beta1
      schema:
        openAPIV3Schema:
          properties:
            spec:
              forProvider:
                properties:
                  region: {type: string}
                  tags:
                    type: object
                    additionalProperties: {type: string}
```

bucketpolicy-crd.yaml — ✗ taggable = false

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: bucketpolicies.s3.aws.upbound.io
  ownerReferences:
    - kind: ManagedResourceDefinition
spec:
  group: s3.aws.upbound.io
  names: {kind: BucketPolicy}
  versions:
    - name: v1beta1
      schema:
        openAPIV3Schema:
          properties:
            spec:
              forProvider:
                properties:
                  bucket: {type: string}
                  policy: {type: string}
                  region: {type: string}
                  # no tags field
```

1 List all CRDs, keep the AWS ones
spec.group must contain .aws.upbound.io – everything else is ignored

2 Taggable only if the schema has forProvider.tags
spec.properties.forProvider.properties.tags exists → kind is taggable (metric crdInfo{taggable="true|false"})

On delete: checked again

MR is terminating and still has gong.io/orphan-tagger

– Non-taggable kind
log, remove the finalizer only – no AWS call; metric non_taggable_mr_terminating

↻ Throttling / other tag error
keep the finalizer – retry on the next poll

taggable = the CRD exposes a tags field

Orphan tagger: never lose an orphaned AWS resource

Kyverno Mutate adds the finalizer – the orphan-tagger-watcher tags what AWS can tag, and always lets the MR go

